

Module 3

Python BDD RESTful Automations — requests, API Testing & Behave

Module 3 of 4 — Selenium / QA Automation Training
Detailed Curriculum Report with Explanations, Examples & Code

This module moves from UI automation to API automation: testing the REST services that power modern web and mobile applications directly, using Python's `requests` library. It then layers Behavior-Driven Development (BDD) on top using the Behave framework so tests can be written in plain-English Gherkin syntax.

19.1 Python Basics for API Automation

REST vs SOAP

Aspect	REST	SOAP
Format	JSON (mostly), also XML	XML only
Protocol	Uses HTTP as the protocol itself	Protocol-independent (can run over HTTP, SMTP...)
Style	Resource-based (nouns), uses HTTP verbs (GET, POST, PUT, DELETE)	Method-based (RPC style)
Contract	No strict contract; OpenAPI/Swagger optional	Strict WSDL contract required
Performance	Lightweight, faster, cache-friendly	Heavier due to XML envelope and strict standards

The Python requests Library

Installation

```
pip install requests
```

requests_basics.py

```
import requests

response = requests.get("https://reqres.in/api/users/2")
print(response.status_code)
print(response.json())
```

Python & Behave Setup

Installing Behave

```
pip install behave
behave --version
```

A typical Behave project layout:

Project structure

```
project/
  features/
    api_login.feature
  steps/
    api_login_steps.py
  environment.py
```

19.2 Python HTTP Libraries

GET Requests & Reading JSON Responses

get_request.py

```
response = requests.get("https://reqres.in/api/users?page=2")
data = response.json()
print(data["data"][0]["email"])
```

Basic Features of requests

- `requests.get/post/put/patch/delete()` for all HTTP verbs.
- Automatic JSON encoding/decoding via `json=` parameter and `.json()`.
- Built-in query-string, header, timeout, and file-upload handling.
- Session objects for connection reuse and persistent cookies/auth.

Validating Response Status Codes and Headers

validate_response.py

```
response = requests.get("https://reqres.in/api/users/2")
assert response.status_code == 200
assert response.headers["Content-Type"].startswith("application/json")
```

POST Requests with Payload and Headers

post_request.py

```
payload = {"name": "Pratik", "job": "QA Automation Engineer"}
headers = {"Content-Type": "application/json"}

response = requests.post("https://reqres.in/api/users", json=payload, headers=headers)
assert response.status_code == 201
print(response.json())
```

End-to-End API Automation Flow

e2e_api_flow.py

```
# 1. Create a user
create = requests.post("https://reqres.in/api/users", json={"name": "Pratik", "job": "QA"})
user_id = create.json()["id"]

# 2. Fetch the user
get = requests.get(f"https://reqres.in/api/users/{user_id}")

# 3. Update the user
update = requests.put(f"https://reqres.in/api/users/{user_id}", json={"job": "Lead QA"})

# 4. Delete the user
delete = requests.delete(f"https://reqres.in/api/users/{user_id}")
assert delete.status_code == 204
```

19.3 API Automation

Global Configuration Object

config.py

```
# config.py
class Config:
    BASE_URL = "https://reqres.in/api"
    TIMEOUT = 10
    DEFAULT_HEADERS = {"Content-Type": "application/json"}

# usage
from config import Config
requests.get(f"{Config.BASE_URL}/users/2", timeout=Config.TIMEOUT)
```

Reusable Payload/Data from External Sources

reusable_payload.py

```
import json

def load_payload(name):
    with open(f"payloads/{name}.json") as f:
        return json.load(f)

payload = load_payload("create_user")
requests.post(f"{Config.BASE_URL}/users", json=payload)
```

Authenticating APIs

auth.py

```
# Basic Auth
requests.get(url, auth=("username", "password"))

# Bearer token
headers = {"Authorization": "Bearer <token>"}
requests.get(url, headers=headers)

# API key in header
headers = {"x-api-key": "abcd1234"}
requests.get(url, headers=headers)
```

Session Management

A Session object persists cookies, headers, and TCP connections across multiple requests, closely mirroring how a real browser/client maintains a login session.

session.py

```

session = requests.Session()
session.headers.update({"Authorization": "Bearer <token>"})

login = session.post(f"{Config.BASE_URL}/login", json={"email": "eve.holt@reqres.in", "password": "pistol05"})
# subsequent calls automatically reuse cookies/headers from the session
profile = session.get(f"{Config.BASE_URL}/users/2")

```

Sending & Managing Cookies

cookies.py

```

response = requests.get(url, cookies={"session_id": "abc123"})
print(response.cookies.get_dict())

```

Setting Up MySQL & Driving Data from Python

Install MySQL connector

```

pip install mysql-connector-python

```

mysql_data.py

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root", password="pass", database="qa_test_db"
)
cursor = conn.cursor()
cursor.execute("SELECT username, password FROM test_users")
for username, password in cursor.fetchall():
    print(username, password)
conn.close()

```

PUT, PATCH, DELETE Methods

put_patch_delete.py

```

# PUT - replaces the entire resource
requests.put(f"{Config.BASE_URL}/users/2", json={"name": "Pratik", "job": "Senior QA"})

# PATCH - updates only the given field(s)
requests.patch(f"{Config.BASE_URL}/users/2", json={"job": "Senior QA"})

# DELETE
requests.delete(f"{Config.BASE_URL}/users/2")

```

Note: PUT expects the complete resource representation and replaces it; PATCH sends only the fields that changed. Using PUT for a partial update can unintentionally null out omitted fields on some APIs.

19.4 Handling HTTP Responses

Response Objects & Reading Content

response_object.py

```
response = requests.get(url)
print(response.status_code)
print(response.headers)
print(response.text)           # raw text
print(response.json())         # parsed JSON (dict/list)
print(response.content)        # raw bytes
```

Handling Binary and Text Data

binary_data.py

```
# Downloading a binary file (e.g. an image or PDF returned by an API)
response = requests.get("https://example.com/file.pdf")
with open("downloaded.pdf", "wb") as f:
    f.write(response.content)
```

Status Codes, Success & Error Handling

Range	Meaning
2xx	Success (200 OK, 201 Created, 204 No Content)
3xx	Redirection (301 Moved Permanently, 304 Not Modified)
4xx	Client error (400 Bad Request, 401 Unauthorized, 404 Not Found)
5xx	Server error (500 Internal Server Error, 503 Service Unavailable)

Exception Handling in requests

exception_handling.py

```
from requests.exceptions import ConnectionError, Timeout, HTTPError

try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()      # raises HTTPError for 4xx/5xx
except Timeout:
    print("Request timed out")
except ConnectionError:
    print("Failed to connect to server")
except HTTPError as e:
    print(f"HTTP error occurred: {e}")
```

Streaming Large Responses

streaming.py

```
with requests.get(url, stream=True) as response:
    with open("large_file.zip", "wb") as f:
        for chunk in response.iter_content(chunk_size=8192):
            f.write(chunk)
```

Handling Cookies and Sessions (Advanced)

cookies_sessions.py

```
session = requests.Session()
response = session.get(url)
print(session.cookies.get_dict())
# cookies persist automatically for every subsequent call on 'session'
```

19.5 Behavior-Driven Development (BDD)

BDD bridges the communication gap between business stakeholders and engineers by describing tests in plain, structured English (Gherkin) that both sides can read. In the Python ecosystem, **Behave** is the leading BDD tool (an alternative is **pytest-bdd**).

Gherkin Syntax & Feature Files

features/api_login.feature

```
# features/api_login.feature
Feature: User Login API

  Scenario: Successful login with valid credentials
    Given the API base URL is set
    When I send a POST request to "/login" with valid credentials
    Then the response status code should be 200
    And the response should contain a token

  Scenario Outline: Login with invalid credentials
    When I send a POST request to "/login" with username "<username>" and password "<password>"
    Then the response status code should be <status_code>

  Examples:
    | username           | password | status_code |
    | eve.holt@reqres.in | wrong    | 400          |
    |                     | pistol   | 400          |
```

Step Definition Files with Auto-Generated Syntax

features/steps/api_login_steps.py

```
# features/steps/api_login_steps.py
from behave import given, when, then
import requests

BASE_URL = "https://reqres.in/api"

@given('the API base URL is set')
def step_set_base_url(context):
    context.base_url = BASE_URL

@when('I send a POST request to "{endpoint}" with valid credentials')
def step_post_valid(context, endpoint):
    payload = {"email": "eve.holt@reqres.in", "password": "cityslicka"}
    context.response = requests.post(context.base_url + endpoint, json=payload)

@when('I send a POST request to "{endpoint}" with username "{username}" and password "{password}"')
def step_post_creds(context, endpoint, username, password):
    payload = {"email": username, "password": password}
    context.response = requests.post(BASE_URL + endpoint, json=payload)

@then('the response status code should be {status_code:d}')
def step_check_status(context, status_code):
    assert context.response.status_code == status_code

@then('the response should contain a token')
def step_check_token(context):
    assert "token" in context.response.json()
```

Hooks — environment.py

features/environment.py

```
# features/environment.py
def before_all(context):
    context.base_url = "https://reqres.in/api"
    print("Test run starting...")

def before_scenario(context, scenario):
    context.session = __import__("requests").Session()

def after_scenario(context, scenario):
    context.session.close()

def after_all(context):
    print("Test run complete.")
```

Parameterization with Scenario Outline

The Scenario Outline + Examples keywords (shown in the feature file above) let one scenario definition run once per data row — equivalent to PyTest's `@parametrize`.

Tagging Mechanism

Tags in a feature file

```
# In the feature file
@smoke
Scenario: Successful login with valid credentials
...

@regression
Scenario Outline: Login with invalid credentials
...
```

Running tagged scenarios

```
behave --tags=smoke
behave --tags=~wip      # run everything except @wip
```

Best Practices for Reusable Step Definitions

- Keep step text generic and parameterized rather than hard-coding values.
- Store request payloads and expected responses in separate JSON files, not inline in code.
- Use context to pass data between steps, not global variables.
- One assertion focus per Then step for clear failure diagnostics.
- Group related steps into logically named step files (auth_steps.py, user_steps.py).

Integrating Tags & Hooks — Generalized BDD Structure

Generalized BDD project layout

```
project/
  features/
    api_login.feature
    api_users.feature
    environment.py
    steps/
      common_steps.py
      auth_steps.py
      user_steps.py
  config.py
  behave.ini
```

Generating Allure Reports for Behave

Allure reporting

```
pip install allure-behave
behave -f allure_behave.formatter:AllureFormatter -o allure-results
allure serve allure-results
```

Module 3 Summary: You can now automate REST APIs end-to-end with Python's requests library (GET, POST, PUT, PATCH, DELETE), handle authentication, sessions, cookies, and streaming, and wrap all of it in readable Gherkin scenarios using Behave, complete with hooks, tags, parameterization, and Allure reporting. Module 4 shifts to a keyword-driven alternative: the Robot Framework.